Light

FDE Cycle & Systems Software

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Hardware & software
2. The FDE cycle
3. Systems software
4. Languages & translators
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **FDE Cycle & Systems Software (8525 Paper 2)**. This topic covers hardware and software definitions, the fetch-decode-execute cycle, system and application software, operating system functions, utility programs, programming language levels, and translators (compilers, interpreters, assemblers).
Exam questions typically ask you to **define** hardware and software, **describe** the stages of the FDE cycle, **explain** the purpose of operating systems and utility programs, **compare** low-level and high-level languages, and **explain** the differences between compilers, interpreters, and assemblers. This is Paper 2 content (section 3.4).

Why the FDE cycle & systems software matters

Every instruction your computer carries out — from loading a web page to running a game — follows exactly the same process inside the CPU: **fetch** the instruction, **decode** it, then **execute** it. Understanding this cycle reveals how all software, from simple calculators to entire operating systems, actually runs on hardware.
Systems software sits between the user and the hardware, managing resources and providing a platform for applications. Without it, every programmer would need to control the hardware directly — an impossibly complex task.

- **Hardware & software:** The physical components and the programs that run on them
- **FDE cycle:** The continuous process that drives all computation
- **Systems software:** Operating systems and utilities that manage the computer
- **Translators:** The tools that convert human-readable code into machine code

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain each point clearly.

- **Define** the terms hardware and software and understand the relationship between them.
- **Explain** the role and operation of the major CPU components: arithmetic logic unit (ALU), control unit, clock, registers, and buses.
- **Explain** the effect of clock speed, number of processor cores, and cache size on CPU performance.
- **Understand** and **explain** the Fetch-Decode-Execute cycle.
- **Explain** what is meant by system software and application software, and give examples of both.
- **Understand** the need for, and functions of, operating systems (OS) and utility programs.
- **Understand** that the OS handles management of processor(s), memory, I/O devices, applications, and security.
- **Know** that there are different levels of programming language: low-level and high-level.
- **Know** that machine code and assembly language are low-level languages and **explain** the differences between them.
- **Understand** the advantages and disadvantages of low-level versus high-level language programming.
- **Understand** that there are three common types of translator: interpreter, compiler, and assembler.
- **Explain** the main differences between these three translators and when each is appropriate.

> **Specification note.** This topic draws primarily from AQA 8525 specification sections 3.4.1 (Hardware and software), 3.4.3 (Software classification), 3.4.4 (Programming languages and translators), and the FDE and CPU component content from 3.4.5 (Systems architecture). Note that memory types, secondary storage, and embedded systems are covered on separate topic pages.

Toolkit

Key definitions

Term	Definition (exam answer)
Hardware	The physical components of a computer system (e.g. CPU, monitor, keyboard, RAM)
Software	Programs (sets of instructions) that tell the hardware what to do

Term	Definition (exam answer)
FDE cycle	The continuous process where the CPU fetches an instruction from memory, decodes it, and then executes it
System software	Software that manages the computer system resources and provides a platform for application software to run (e.g. operating systems, utility programs)
Application software	Software that performs end-user tasks (e.g. word processor, web browser, game)
Operating system	System software that manages the processor(s), memory, I/O devices, applications, and security
Compiler	A translator that converts an entire high-level language program into machine code in one go, producing an executable file
Interpreter	A translator that reads and executes a high-level language program one line at a time, without producing a separate machine code file
Assembler	A translator that converts assembly language into machine code; each assembly instruction maps to one machine code instruction

The Fetch-Decode-Execute cycle

1️⃣ **Fetch**
The next instruction is fetched from main memory to the CPU

2️⃣ **Decode**
The instruction is decoded to work out what it is

3️⃣ **Execute**
The instruction is carried out (may include reading/writing from/to main memory)

This cycle repeats continuously — the CPU fetches, decodes, and executes instructions one after another for as long as the computer is running.

CPU components involved in the FDE cycle

Component	Role in the FDE cycle
Control Unit (CU)	Directs the FDE cycle; fetches instructions, decodes them, and sends control signals to other components to execute them
Arithmetic Logic Unit (ALU)	Performs calculations (arithmetic) and logical comparisons during the execute stage
Clock	Generates regular electrical pulses (ticks) that synchronise the FDE cycle; one tick = one cycle
Registers	Tiny, ultra-fast storage locations inside the CPU that hold the current instruction, data being processed, and memory addresses during the cycle
Buses	Collections of wires that transmit data, addresses, and control signals between the CPU and main memory

Operating system — five management functions

Processor
Allocates CPU time to running programs (multitasking)

Memory
Allocates and manages RAM for programs; transfers data to/from storage

I/O Devices
Manages communication with peripherals via drivers

Applications
Provides a platform to install, run, and close applications

Security
Manages user accounts, access rights, passwords, and authentication

Translator comparison

Feature	Compiler	Interpreter	Assembler
Input language	High-level	High-level	Assembly

Feature	Compiler	Interpreter	Assembler
Output	Produces machine code executable	Does NOT produce machine code directly	Produces machine code
Translation method	Translates the whole program at once before execution	Translates and executes one line at a time	Translates the whole program; 1:1 mapping
Speed of execution	Fast — compiled code runs directly	Slower — must translate each line every time	Fast — machine code runs directly
Error reporting	Reports all errors after compilation	Stops at the first error (easier for debugging)	Reports errors after assembly
Distribution	Executable can be distributed without source code	Source code must be distributed (interpreter needed to run)	Machine code can be distributed without source

1. Hardware and software

Every computer system is made up of two fundamental parts: **hardware** and **software**. Understanding the relationship between them is the starting point for the entire computer systems topic.

1.1 Definitions

Hardware

- The **physical components** of a computer system
- You can see and touch hardware
- Examples: CPU, RAM, hard drive, monitor, keyboard, mouse, motherboard

Software

- **Programs** — sets of instructions that tell the hardware what to do
- Software is intangible (you cannot physically touch it)
- Examples: Windows, Google Chrome, Microsoft Word, Python

1.2 The relationship

Hardware and software depend on each other:

- **Hardware without software** is useless — it has no instructions to follow and cannot perform any tasks
- **Software without hardware** cannot run — there is no physical device to execute the instructions
- Software provides the instructions; hardware carries them out

> **Key distinction:** Hardware is the physical machinery. Software is the set of instructions that controls the hardware. Neither is useful without the other.

2. The Fetch-Decode-Execute cycle

The **Fetch-Decode-Execute (FDE) cycle** is the fundamental process by which the CPU processes every instruction in a program. The cycle repeats continuously, billions of times per second in modern processors.

2.1 The three stages

Stage 1 — Fetch

The **next instruction** is fetched from **main memory** (RAM) and transferred to the CPU.

- The CPU checks the address of the next instruction to fetch
- The instruction is copied from main memory to the CPU via the bus
- The address counter moves on to point to the next instruction

Stage 2 — Decode

The **control unit** decodes the instruction to work out **what it is** and what needs to happen.

- The instruction is split into its component parts (operation and data)
- The control unit identifies the type of operation to perform
- The control unit prepares the necessary components (e.g. the ALU for a calculation)

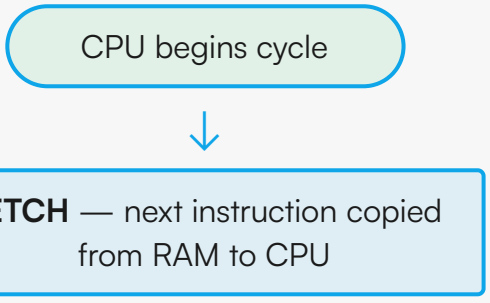
Stage 3 — Execute

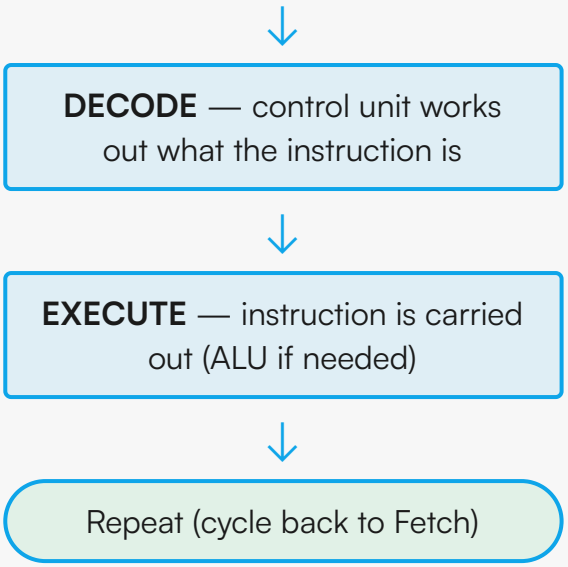
The instruction is **carried out**. What happens depends on the type of instruction:

- **Arithmetic/logic:** The ALU performs the calculation or comparison
- **Data transfer:** Data is moved to/from main memory or a register
- **Branch:** The address counter is changed to point to a different instruction

This may include reading data from or writing data to main memory.

FDE cycle — visual summary





2.2 CPU components in the FDE cycle

The major components of the CPU each play a specific role during the FDE cycle:

- **Control Unit (CU):** The "director" — it coordinates the entire cycle, fetches and decodes instructions, and sends control signals to the ALU, registers, and memory
- **Arithmetic Logic Unit (ALU):** Performs all calculations (addition, subtraction, multiplication, division) and logical comparisons (greater than, equal to, less than) during the execute stage
- **Clock:** Generates regular electrical pulses that synchronise each step of the FDE cycle. The faster the clock, the more cycles per second
- **Registers:** Tiny, ultra-fast storage locations inside the CPU that temporarily hold the current instruction, data being processed, and the address of the next instruction. Much faster than RAM
- **Buses:** Collections of wires that carry data, addresses, and control signals between the CPU and main memory (and other components)

> **AQA note:** Knowledge of specific named registers (e.g. Program Counter, MAR, MDR, Accumulator) is NOT required at GCSE. You just need to know that registers exist and that they are small, fast storage locations inside the CPU.

2.3 Factors affecting CPU performance

Three factors affect how fast a CPU can process instructions:

Clock speed

Measured in **hertz (Hz)** — the number of FDE cycles per second.
A 3 GHz processor performs approximately 3 billion cycles per second.
Higher clock speed = faster processing

Number of cores

Each **core** is an independent processing unit that can carry out its own FDE cycle.
A quad-core CPU has 4 cores that can process 4 instructions simultaneously.
More cores = more tasks at once

Cache size

Cache is a small amount of very fast memory inside the CPU.
Frequently used data is stored in cache so the CPU doesn't have to fetch it from slower RAM.
More cache = fewer slow memory accesses

Worked Example 1 — Describing the FDE cycle

Question: Describe the three stages of the Fetch-Decode-Execute cycle. [3 marks]

Model answer:
Fetch: The next instruction is fetched to the CPU from main memory **(1)**.
Decode: The instruction is decoded to work out what it is **(1)**.
Execute: The instruction is executed/carried out, which may include reading or writing data from/to main memory **(1)**.

Exam tip: Use the exact AQA phrasing: "fetched to the CPU from main memory", "decoded to work out what it is", "executed (carried out)". These map directly to mark scheme points.

Worked Example 2 — CPU components

Question: Explain the role of the ALU and the control unit within the CPU. [4 marks]

Model answer:
The **ALU (Arithmetic Logic Unit)** performs arithmetic operations such as addition and subtraction **(1)** and logical comparisons such as checking whether one value is greater than another **(1)**.
The **control unit** coordinates and manages the FDE cycle **(1)** by fetching instructions, decoding them, and sending control signals to other components to execute them **(1)**.

Exam tip: For the ALU, always mention BOTH arithmetic (calculations) AND logic (comparisons). Many students only mention one and lose a mark.

Worked Example 3 — CPU performance factors

Question: Explain how increasing the clock speed affects the performance of a CPU. [2 marks]

Model answer:
Increasing the clock speed means more FDE cycles are completed per second **(1)**, so more instructions are processed in the same amount of time, making the CPU faster **(1)**.

Exam tip: Link the factor to the FDE cycle. Clock speed = more cycles per second. Cores = more simultaneous cycles. Cache = fewer slow fetches from RAM.

3. Systems software

Software is classified into two categories: **system software** and **application software**. Understanding the difference is a common exam question.

3.1 System software vs application software

System software

- Manages the computer system resources
- Provides a platform for application software to run
- Runs "behind the scenes"
- **Examples:** operating systems (Windows, macOS, Linux), utility programs (antivirus, disk defragmenter, backup software)

Application software

- Performs **end-user tasks**
- Designed for a specific purpose the user wants to achieve
- Runs on top of the operating system
- **Examples:** word processors, web browsers, games, spreadsheets, email clients, photo editors

Key distinction: System software manages the computer itself. Application software helps the user complete a task. The operating system is system software — it is NOT an application.

3.2 The operating system

The **operating system (OS)** is the most important piece of system software. It acts as an intermediary between the user, application software, and the hardware. AQA requires you to know five management functions:

1. Processor management

The OS decides which programs get access to the CPU and for how long. In a multitasking system, the OS rapidly switches between programs so they appear to run simultaneously. It allocates processor time fairly between all active processes.

2. Memory management

The OS allocates RAM to each running program and ensures programs do not interfere with each other's memory. When a program is opened, the OS loads it from secondary storage into RAM. When a program closes, its memory is freed for other programs.

3. I/O device management

The OS manages communication between the computer and peripheral devices (printers, keyboards, monitors, etc.) using **device drivers**. Drivers are specialised programs that translate the OS's generic instructions into commands the specific device can understand.

4. Application management

The OS provides a platform for applications to be installed, run, and closed. It manages the loading of applications from secondary storage into memory and provides common services (such as file access and screen output) that applications can use.

5. Security management

The OS manages user accounts, passwords, and access rights. It controls which users can access which files and resources, and provides authentication to verify user identity before granting access to the system.

3.3 Utility programs

Utility programs are a type of system software designed to help maintain, optimise, or protect the computer system. They perform specific housekeeping tasks.

Utility	Purpose
Antivirus	Detects, quarantines, and removes malware to protect the system
Disk defragmenter	Reorganises fragmented files on a magnetic hard drive so they are stored in contiguous blocks, improving read speed
Backup	Creates copies of files so they can be restored if the originals are lost or corrupted
File management	Allows users to organise, move, copy, rename, and delete files and folders

Utility	Purpose
Compression	Reduces file sizes to save storage space or speed up file transfer
Encryption	Scrambles data so it can only be read by someone with the correct decryption key

Worked Example 4 — System vs application software

Question: Explain the difference between system software and application software. Give one example of each. [4 marks]

Model answer:
System software manages the computer system's resources and provides a platform for application software to run **(1)**. An example is the operating system (e.g. Windows) **(1)**.
Application software performs end-user tasks **(1)**. An example is a word processor (e.g. Microsoft Word) **(1)**.

Exam tip: Never give the operating system as an example of application software. This is a very common mistake and will always cost marks.

Worked Example 5 — Operating system functions

Question: Describe two functions of an operating system. [4 marks]

Model answer:
Memory management: The operating system allocates RAM to running programs **(1)** and ensures that programs do not access each other's allocated memory space **(1)**.
Processor management: The operating system decides which programs get access to the CPU **(1)** and allocates processor time so multiple programs can appear to run simultaneously (multitasking) **(1)**.

Exam tip: For each function, give the WHAT (what the OS does) and the WHY (why it matters or what would happen without it). Two well-developed points beat four shallow ones.

4. Programming languages and translators

Programs are written by humans in languages they can understand, but processors can only execute **machine code** (binary). Translators bridge this gap.

4.1 Levels of programming language

High-level languages

- Close to human language (English-like keywords)
- One high-level statement may map to many machine code instructions
- **Portable:** can run on different types of processor (once translated)
- Easier to read, write, and debug
- **Examples:** Python, Java, C#, VB.NET

Low-level languages

- Close to the hardware / machine code
- Two types: **machine code** and **assembly language**
- **Not portable:** specific to a processor or processor family
- Harder to read, write, and debug
- Give direct control over hardware

4.2 Machine code and assembly language

Both machine code and assembly language are classified as low-level languages, but they are not the same thing:

Feature	Machine code	Assembly language
Format	Binary (0s and 1s)	Mnemonics (short text codes like `ADD`, `MOV`, `LDA`)
Readability	Virtually unreadable by humans	More readable than machine code, but still difficult
Relationship	Executed directly by the processor	Has a **1:1 correspondence** with machine code — each assembly instruction maps to exactly one machine code instruction
Processor specific?	Yes — each processor type has its own instruction set	Yes — tied to the same processor's instruction set
Typical use	What the processor actually runs	Embedded systems, device drivers, direct hardware control

> **Key point for exams:** Assembly language has a **1:1 correspondence** with machine code. This means each line of assembly language corresponds to exactly one machine code instruction. This is a very commonly tested fact.

4.3 Why use high-level languages?

Most programs are written in high-level languages. The AQA specification expects you to explain why:

- **Easier to learn and use:** English-like syntax means programmers can write code faster
- **Easier to read and debug:** Code is more understandable, so errors are easier to find
- **Portable:** The same code can be translated and run on different types of processor
- **One statement = many instructions:** A single high-level statement can replace dozens of machine code instructions, making development faster

4.4 Advantages of low-level languages

Despite being harder to use, low-level languages have important advantages:

- **Direct hardware control:** The programmer can manipulate specific memory locations and hardware registers
- **Efficient code:** Programs can be optimised for speed and minimal memory usage
- **Ideal for embedded systems:** Limited resources mean every byte matters
- **No translator overhead:** Machine code runs directly on the processor without needing compilation or interpretation

4.5 Translators

All code written in high-level or assembly languages must be **translated** into machine code before the processor can execute it. There are three types of translator:

Compiler

Translates an **entire** high-level language program into machine code in one go, producing a standalone executable file.

- The source code is translated all at once before the program runs
- The output is an executable file that can be run repeatedly without re-translating
- Errors are reported after the whole program is compiled (not during execution)
- The executable can be distributed without the source code — protects intellectual property
- Compiled programs run **faster** because translation has already happened

Interpreter

Translates and executes a high-level language program **one line at a time**.

- Each line is translated then immediately executed before moving to the next
- Does **NOT** produce a separate machine code file — it calls machine code subroutines within its own code to carry out each statement
- Stops at the first error it encounters, making it useful for debugging and development
- The source code must be present every time the program runs (the interpreter is needed)
- Programs run **slower** because each line must be translated every time the program runs

Assembler

Translates assembly language into machine code.

- Each assembly instruction is converted into exactly **one** machine code instruction (1:1 mapping)

- Produces a machine code executable, similar to a compiler
- Specific to the processor — a different assembler is needed for each processor type

Critical AQA distinction: Assemblers and compilers translate their input into machine code **directly**. Interpreters do **NOT** generate machine code directly — they call appropriate machine code subroutines within their own code to carry out statements. This is a key mark scheme point.

4.6 When to use each translator

Situation	Best translator	Reason
Distributing finished software to customers	**Compiler**	Produces a fast executable; source code stays hidden
Developing and testing a program	**Interpreter**	Stops at the first error, making debugging easier; no need to recompile after every change
Writing software for a specific embedded device	**Assembler**	Assembly gives direct hardware control needed for embedded systems; assembler converts to machine code for the target processor

Worked Example 6 — High-level vs low-level

Question: Explain two advantages of using a high-level language instead of a low-level language. [4 marks]

Model answer:
High-level languages are easier to read and write because they use English-like keywords **(1)**, which means programs can be developed more quickly and with fewer errors **(1)**.
High-level language code is portable, meaning it can be run on different types of processor **(1)**, whereas low-level code is specific to one processor type and would need to be rewritten for a different processor **(1)**.

Exam tip: Always develop your points. Don't just say "easier to read" — explain the consequence: faster development, fewer errors, easier debugging.

Worked Example 7 — Compiler vs interpreter

Question: Explain two differences between a compiler and an interpreter. [4 marks]

Model answer:
A compiler translates the entire source code into machine code in one go before execution **(1)**, whereas an interpreter translates and executes the code one line at a time **(1)**.
A compiler produces a standalone executable file that can be run without the source code **(1)**, whereas an interpreter does not produce a separate executable — the source code and interpreter must be present each time the program is run **(1)**.

Exam tip: For comparison questions, structure each point as "A does X, whereas B does Y." This makes the comparison explicit and guarantees both sides are covered.

Worked Example 8 – Assembly language

Question: State what is meant by the term "assembly language" and explain the role of an assembler. [3 marks]

Model answer:
Assembly language is a low-level programming language that uses mnemonics (short text codes) to represent machine code instructions **(1)**. It has a 1:1 correspondence with machine code, meaning each assembly instruction maps to exactly one machine code instruction **(1)**. An assembler is a translator that converts assembly language into machine code so it can be executed by the processor **(1)**.

Exam tip: The phrase "1:1 correspondence" is a guaranteed mark scheme point when discussing assembly language. Always include it.

Worked Example 9 – Interpreters and machine code

Question: Explain why an interpreter does not produce machine code directly. [2 marks]

Model answer:
An interpreter does not generate a separate machine code file **(1)**. Instead, it reads each line of source code and calls appropriate machine code subroutines within its own code to carry out the statements **(1)**.

Exam tip: This is the precise AQA wording. The interpreter has its own machine code subroutines built in, and it calls these as needed — it does NOT

translate the user's code into machine code.

Worked examples

The worked examples are embedded throughout the sections above. Here is a summary of all examples and where to find them:

#	Topic	Skill tested	Marks
1	The FDE cycle stages	Describe	3
2	ALU and control unit roles	Explain	4
3	CPU performance (clock speed)	Explain	2
4	System vs application software	Explain + examples	4
5	Operating system functions	Describe	4
6	High-level vs low-level advantages	Explain	4
7	Compiler vs interpreter	Explain differences	4
8	Assembly language and assembler	State and explain	3
9	Interpreters and machine code	Explain	2
10	Hardware and software definitions	Define	2
11	Utility programs	Describe	4
Ext.	Why high-level languages dominate modern development	Explain (extended response)	6

Worked Example 10 — Defining hardware and software

Question: Define the terms hardware and software. [2 marks]

Model answer:
Hardware is the physical components of a computer system **(1)**.
Software is programs / sets of instructions that tell the hardware what to do **(1)**.

Exam tip: "Physical" is the key word for hardware. "Instructions/programs" is the key word for software. Keep definitions concise.

Worked Example 11 — Utility programs

Question: Describe the purpose of two different utility programs. [4 marks]

Model answer:
Antivirus software scans the computer for malware **(1)** and quarantines or removes any malicious software it detects to protect the system **(1)**.
Disk defragmenter reorganises files stored on a magnetic hard drive so that file fragments are stored in contiguous (next to each other) blocks **(1)**, which reduces the time the read/write head needs to access the data, improving performance **(1)**.

Exam tip: For each utility, give the PURPOSE and the BENEFIT. "Antivirus removes malware" is one mark. "...to protect the system from damage or data loss" is the second.

Worked Example Ext. — Extended response

Question: Explain why most programs are written in high-level languages rather than low-level languages. In your answer, discuss the role of translators. [6 marks]

Model answer:
Most programs are written in high-level languages because they use English-like syntax that is much easier for programmers to read, write, and understand **(1)**. This means programs can be developed faster and with fewer errors, reducing development time and cost **(1)**.
High-level language code is also portable, meaning the same source code can be translated and run on different types of processor **(1)**. In contrast, low-level code such as machine code is specific to one processor type, so a program written for one processor would need to be completely rewritten for another **(1)**.
Because processors can only execute machine code, high-level programs must be translated. A compiler translates the entire program into a machine code executable that can be distributed and run efficiently **(1)**. Alternatively, an interpreter can translate and execute the code one line at a time during development, which is useful for testing and debugging because it stops at the first error encountered **(1)**.

Exam tip: Extended-response questions require developed, connected points — not just a list. Structure your answer: advantages of high-level → disadvantages of low-level → how translators bridge the gap. Use connective language: "This means...", "In contrast...", "Because of this...".

Quick reference

Key definitions (mark-scheme style)

Term	Definition
Hardware	The physical components of a computer system.
Software	Programs (sets of instructions) that tell the hardware what to do.
System software	Software that manages the computer's resources and provides a platform for application software to run.
Application software	Software that performs end-user tasks.
Operating system	System software that manages the processor(s), memory, I/O devices, applications, and security.
Utility program	System software designed to help maintain, optimise, or protect the computer system.
FDE cycle	The continuous cycle where the CPU fetches an instruction from memory, decodes it, and executes it.
High-level language	A programming language that uses English-like syntax and is portable across different processor types.
Low-level language	A programming language close to machine code; includes machine code (binary) and assembly language (mnemonics).
Machine code	Binary instructions that the processor can execute directly; specific to a processor or processor family.
Assembly language	A low-level language that uses mnemonics with a 1:1 correspondence to machine code instructions.
Compiler	Translates an entire high-level program into machine code at once, producing an executable file.
Interpreter	Translates and executes a high-level program one line at a time; does not produce a machine code file.
Assembler	Translates assembly language into machine code; one assembly instruction becomes one machine code instruction.

Translator comparison summary

	Compiler	Interpreter	Assembler
Translates	High-level → machine code	High-level → (executed via internal subroutines)	Assembly → machine code

	Compiler	Interpreter	Assembler
Method	Whole program at once	Line by line	Whole program; 1:1 mapping
Output	Executable file	No separate file	Executable file
Execution speed	Fast	Slower	Fast
Errors	Reported after compilation	Stops at first error	Reported after assembly
Best for	Distributing finished software	Development and testing	Embedded systems, hardware control

High-level vs low-level at a glance

Feature	High-level	Low-level
Readability	Easy — English-like	Difficult — binary or mnemonics
Portability	Portable across processors	Processor-specific
Development speed	Faster	Slower
Hardware control	Limited	Direct control
Code efficiency	Less optimised	Highly optimised
Debugging	Easier	Harder

Typical pitfalls

Pitfall 1 — Saying "fetch-execute" instead of "fetch-decode-execute".
Wrong: "The CPU fetches and executes instructions."
Right: The cycle has THREE stages: fetch, decode, and execute. Missing "decode" loses a mark every time.

Pitfall 2 — Confusing system software and application software.
Wrong: "The operating system is an application."
Right: The OS is system software. Application software performs end-user tasks. System software manages the computer's resources.

Pitfall 3 — Saying interpreters produce machine code.
Wrong: "An interpreter translates code into machine code one line at a time."

Right: An interpreter does NOT produce machine code. It calls machine code subroutines within its own code to carry out each statement.

Pitfall 4 — Confusing assembly language and machine code.
Wrong: "Assembly language and machine code are the same thing."
Right: Machine code is binary (0s and 1s). Assembly language uses mnemonics (e.g. ADD, MOV). Assembly has a 1:1 correspondence with machine code, but they are different representations.

Pitfall 5 — Forgetting the 1:1 correspondence.
Wrong: "Assembly language is translated into machine code."
Right: Assembly language has a **1:1 correspondence** with machine code — each assembly instruction maps to exactly one machine code instruction. This specific phrase scores marks.

Pitfall 6 — Vague answers about CPU performance.
Wrong: "More cache makes it faster."
Right: More cache means frequently used data/instructions can be stored closer to the CPU, so the CPU does not have to fetch them from slower RAM, which reduces the time per FDE cycle.

Pitfall 7 — Not explaining WHY high-level languages are preferred.
Wrong: "High-level languages are better."
Right: Give specific reasons: easier to read/write (English-like syntax), portable (run on different processors), faster development time, easier to debug.

Pitfall 8 — Confusing compiler and assembler.
Wrong: "A compiler converts assembly language."
Right: A compiler translates HIGH-LEVEL language code. An assembler translates ASSEMBLY language code. Both produce machine code, but they work with different input languages.

Pitfall 9 — Incomplete OS function descriptions.
Wrong: "The OS manages memory."
Right: The OS allocates RAM to running programs and ensures that programs do not interfere with each other's memory space. Always say WHAT the OS does and WHY.

Pitfall 10 — Thinking "hardware" means only internal components.
Wrong: "Hardware is the CPU and RAM."
Right: Hardware is ALL physical components — including peripherals like monitors, keyboards, mice, and printers. If you can touch it, it's hardware.

Assessment Hub
Long-Answer
Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact · Privacy · Terms · Accessibility · Copyright · DMCA · YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.